Process State Diagram, Schedulers, and Context Switch

Lecture 6 — How the OS Tracks and Switches Processes

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Compute. Think. Click.

Welcome: Who Is Using the CPU Right Now?

The big question

A lab machine has *one* CPU core. On it, `gcc` is compiling a large project, a text editor is waiting for your next keystroke, and a backup job is queued for the night. All three feel "running." At any instant, how many of them actually are — and how does the OS decide, remember, and switch?

- `P1` = `gcc` (long CPU bursts), `P2` = editor (short bursts, mostly waiting), `P3` = backup (submitted, not yet admitted).
- On one core, at most **one** process is running at any instant.

Running thread for today

We follow `P1`, `P2`, and `P3` through their states, the queues that hold them, the schedulers that move them, and the context switches that hand the CPU from one to the next.

Recall from Lecture 5: The Process and Its PCB

- A **process** is a program in execution: code, data, stack, heap, and the CPU state that goes with it.
- The kernel records each process in a **process control block** (PCB): state, `PC`, registers, scheduling information, memory-management information, accounting, and open files.
- The PCB is what lets a process *stop* and later *resume* exactly where it left off.

Today's question

The PCB has a *state* field. Which values can it hold, which events change it, and who changes it?

Silberschatz et al., Ch. 3 §3.1, pp. 101–103.

Roadmap: Four Questions

1. **Which states can a process be in?** — the five-state diagram and its legal transitions.
2. **Where does the kernel keep processes?** — the job, ready, and device queues, and the queueing diagram.
3. **Who moves processes between queues?** — the long-term, short-term, and medium-term schedulers.
4. **What does a switch cost?** — the context switch, the dispatcher, and its overhead.

The states are the vocabulary; the queues are the data structures; the schedulers are the policies; the context switch is the mechanism.

Concepts That Click

A process does not just “run.” It is created, waits its turn, runs, blocks, and finishes — and the kernel tracks every step in one field of the PCB.

The Five Process States

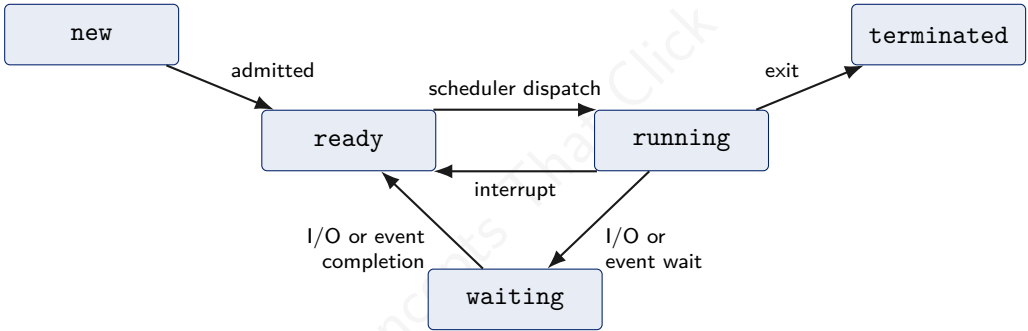
Definition

The **state** of a process is the PCB field that records its current activity. A process is in exactly one of five states at any instant.

`new`	being created; the PCB is being set up
`ready`	in memory, able to run, waiting only for a CPU
`running`	its instructions are being executed on a CPU
`waiting`	blocked until some event occurs (I/O completion, a signal)
`terminated`	finished; its resources are being reclaimed

Silberschatz et al., Ch. 3 §3.1, p. 103. Stallings calls `waiting` "blocked" and `terminated` "exit" (Ch. 3 §3.2, p. 111).

The State Diagram



After Silberschatz et al., Ch. 3 §3.1, p. 103.

Concepts That Click

The Six Legal Transitions

Transition	Triggering event	Who acts
`new` → `ready`	admitted to the system	long-term scheduler
`ready` → `running`	chosen for the CPU	short-term scheduler + dispatcher
`running` → `ready`	timer interrupt / preemption	kernel (interrupt handler)
`running` → `waiting`	I/O request, `wait`, sleep	the process itself (system call)
`waiting` → `ready`	the awaited event occurs	kernel (I/O-completion interrupt)
`running` → `terminated`	`exit` or fatal error	the process / the kernel

Two transitions that never happen

`waiting` → `running` (a woken process must queue for the CPU) and `ready` → `waiting` (only a running process can issue the request it would wait on).

Worked Trace: P2, the Editor, for One Keystroke

1. P2 is `running`; it calls `read` on the keyboard. No key is pending. `running` → `waiting`
2. The kernel places P2's PCB on the keyboard's device queue and runs another process.
3. You press a key. The keyboard interrupt handler finds P2 waiting for it. `waiting` → `ready`
4. P2's PCB joins the ready queue. It does *not* run yet.
5. The short-term scheduler picks P2; the dispatcher loads its context. `ready` → `running`
6. P2 echoes the character and calls `read` again. `running` → `waiting`

Three transitions per keystroke. The editor spends almost all of its life in `waiting`.

Concepts That Click

How Many Processes per State?

- With N CPU cores, at most N processes are `running`.
- Any number may be `ready` or `waiting` at once.
- On our one-core lab machine, at one instant: `P1` `running`, `P2` `waiting` on the keyboard, `P3` `new` (still in the batch queue).

A snapshot of 40 processes on a 4-core machine

At most 4 are `running`. If 9 are `ready`, the remaining $40 - 4 - 9 = 27$ are `waiting` (or `new`/`terminated`). Typical desktops: most processes are waiting most of the time.

Socratic Check 1

Question

`P2` is `waiting` for the disk. The disk interrupt arrives and the CPU is idle. Does `P2` go straight to `running`?

- No — in the diagram it goes `waiting` → `ready`. The interrupt handler only marks it runnable.
- The short-term scheduler then chooses among *all* ready processes; with an idle CPU it may pick `P2` almost immediately.
- Keeping "wake up" and "get the CPU" as separate steps is what lets the scheduler enforce a policy.

Extending the Diagram: Suspended Processes

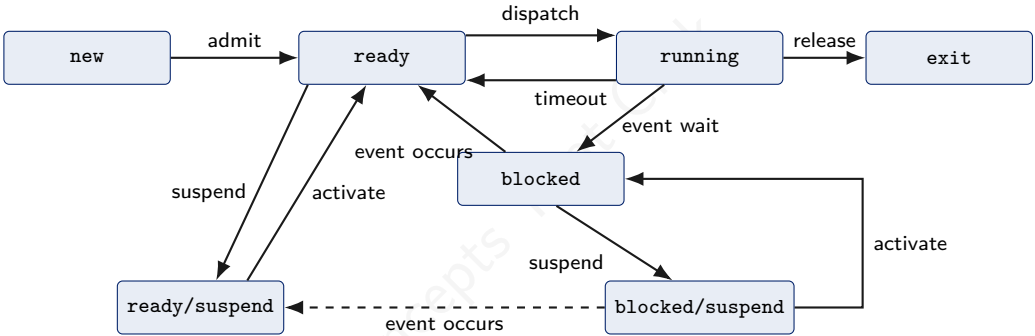
- Memory is finite. If every process in memory is `waiting`, the CPU idles even though more work is queued.
- The OS can **swap out** a process: copy its image to disk and free its memory. The process is now **suspended**.
- Stallings splits each of `ready` and `waiting` into an in-memory and a suspended version, giving a **seven-state model**.

The two new states

`ready/suspend`: on disk, able to run once brought back.
`blocked/suspend`: on disk *and* still waiting for an event.

Stallings, Ch. 3 §3.2, p. 111.

The Seven-State Diagram



After Stallings, Ch. 3 §3.2, p. 111. Top row: in memory. Bottom row: swapped out to disk.

A state is just a label. To act on it, the kernel keeps processes of each state in queues — and the queues are where scheduling happens.

Three Kinds of Queue

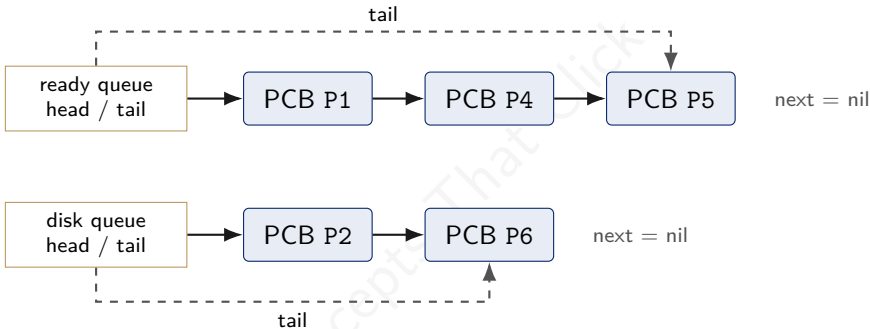
- **Job queue**: every process in the system (in Silberschatz's batch-oriented description, jobs submitted and awaiting admission).
- **Ready queue**: processes in main memory that are `ready` — usually a linked list of PCBs.
- **Device queues**: one per device; processes `waiting` for that device.

Implementation

A queue header holds pointers to the first and last PCB; each PCB has a pointer field to the next PCB in the same queue. Moving a process between queues moves a pointer, not the process.

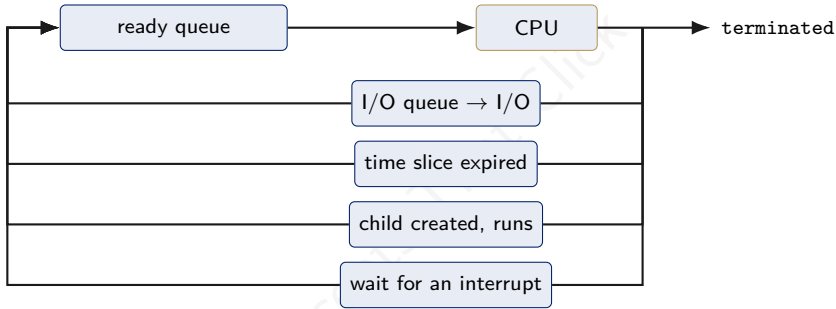
Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

The Ready Queue and Device Queues as Linked Lists



After Silberschatz et al., Ch. 3 §3.2, pp. 105–110. A PCB sits on exactly one of these queues at a time.

The Queueing Diagram



After Silberschatz et al., Ch. 3 §3.2, pp. 105–110. Every path out of the CPU, except termination, leads back to the ready queue.

Four Ways to Leave the CPU

A running process stops running for exactly one of these reasons:

1. It issues an **I/O request** and is placed on a device queue.
2. Its **time slice expires**; a timer interrupt returns it to the ready queue.
3. It **creates a child** and may wait for the child to terminate.
4. It is **removed by an interrupt** or waits for an event.

The three processes

`P2` (editor) leaves by route 1 almost every burst. `P1` (gcc) leaves mostly by route 2. A shell running gcc leaves by route 3.

Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

Worked Trace: Queue Membership Over Time

Time	Event	Running	Ready queue	Keyboard queue
t_0	start	P1	P2	—
t_1	P1's time slice expires	P2	P1	—
t_2	P2 calls `read`	P1	—	P2
t_3	key pressed (interrupt)	P1	P2	—
t_4	P1's time slice expires	P2	P1	—
t_5	P3 admitted	P2	P1, P3	—

- Every row has exactly one running process; each PCB appears in exactly one place.
- At t_3 the interrupt moves P2 to the ready queue *behind* nothing — but P1 keeps the CPU until t_4.

Concepts That Click

Queues hold the processes. Schedulers decide who moves between them — and an OS has up to three of them, working on very different timescales.

Concept Clinic

Three Schedulers, Three Timescales

Scheduler	Moves a process	Runs	Controls
Long-term (job)	job pool → ready queue (`new` → `ready`)	seconds to minutes	degree of multiprogramming
Short-term (CPU)	ready queue → CPU (`ready` → `running`)	every few ms	who runs next
Medium-term	memory ↔ disk (swap out / swap in)	occasionally	memory pressure, mix

Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

Concepts That Click

The Long-Term Scheduler

- Selects processes from the job pool (on disk) and loads them into memory for execution.
- Controls the **degree of multiprogramming** n — the number of processes in memory.
- In a stable system, the average admission rate equals the average departure rate.
- Runs rarely, so it can afford a careful choice.

Where it exists

Batch systems have one. Time-sharing systems such as UNIX and Windows often have *none*: every new process goes straight to the short-term scheduler.

Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

I/O-Bound vs. CPU-Bound Processes

I/O-bound

Spends more time doing I/O than computing; many short CPU bursts. *Example:* `P2`, the editor.

CPU-bound

Spends most time computing; few, long CPU bursts. *Example:* `P1`, `gcc`.

- All I/O-bound: the ready queue is almost always empty; the CPU idles.
- All CPU-bound: device queues are empty; devices idle.
- The long-term scheduler aims for a good **process mix** so both CPU and devices stay busy.

Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

Concepts That

The Short-Term Scheduler

- Selects one process from the ready queue and allocates the CPU to it.
- Runs very often — at least once every time slice (typically every few tens of milliseconds) and after every block.
- Must therefore be **fast**: its own running time is pure overhead.

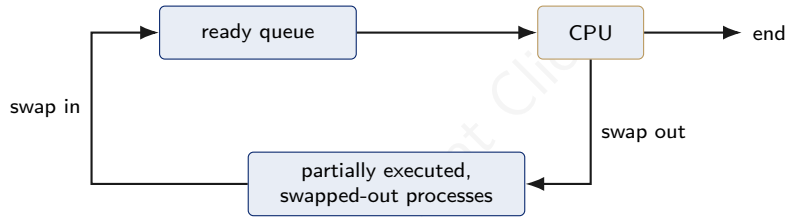
The overhead arithmetic

The scheduler takes 1 ms to decide, then lets a process run for 10 ms. Fraction of CPU spent scheduling:

$$\frac{1}{10+1} \approx 9\%.$$

Silberschatz et al., Ch. 3 §3.2, pp. 105–110. How it chooses (FCFS, SJF, RR, …) is Lectures 8–9.

The Medium-Term Scheduler: Swapping



- Removes a process from memory to reduce n, and later reintroduces it where it left off.
- Used to improve the process mix or to free memory when demand exceeds supply.

After Silberschatz et al., Ch. 3 §3.2, pp. 105–110.

Schedulers on the State Diagram

Scheduler	**Transitions it drives**
Long-term	`new` → `ready` (admit)
Short-term	`ready` → `running` (dispatch); with preemption, also decides `running` → `ready`
Medium-term	`ready` ↔ `ready/suspend`, `blocked` ↔ `blocked/suspend`

Not a scheduler's decision

`running` → `waiting` is caused by the process's own request; `waiting` → `ready` is caused by the completion interrupt. No scheduler chooses those.

Socratic Check 2

Question

Your laptop runs Linux. Students open dozens of browser tabs until the machine slows to a crawl. Which scheduler should have stopped this, and why did none of them?

- Admission control is the long-term scheduler's job, but a time-sharing system typically has *no* long-term scheduler.
- Every new process goes straight to the ready queue; stability depends on the users noticing and quitting programs.
- Memory pressure is instead handled by swapping/paging — a medium-term mechanism — which is why the machine slows rather than refuses.

The short-term scheduler picks the next process. Handing it the CPU means saving one process's context and restoring another's — the context switch.

What Is the Context?

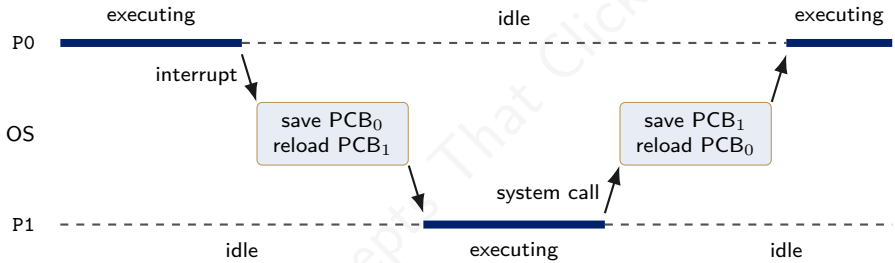
Definition

The **context** of a process is the CPU and kernel state needed to resume it: register values, `PC`, process state, and memory-management information — all represented in its PCB. A **context switch** saves the context of the current process into its PCB and restores the saved context of the process scheduled to run next.

- Two halves: **state save** and **state restore**.
- The switch is triggered by an interrupt, a trap, or a system call that blocks.

Silberschatz et al., Ch. 3 §3.2, p. 110.

The Context Switch on a Timeline



After Silberschatz et al., Ch. 3, pp. 103–110. The gold boxes are pure overhead.

Concepts That Click

The Steps of a Context Switch

1. An interrupt or trap enters the kernel; hardware saves the user `PC` and flags.
2. The kernel saves the remaining registers of `P0` into PCB_0.
3. It updates PCB_0's state (`ready` or `waiting`) and links PCB_0 onto the right queue.
4. The short-term scheduler selects `P1`.
5. The kernel sets PCB_1's state to `running` and switches memory-management state (page-table base, TLB as needed).
6. It restores `P1`'s registers from PCB_1 and returns to user mode at `P1`'s saved `PC`.

Stallings, Ch. 3 §3.4, p. 135 (the full list of process-switch steps); Bovet & Cesati, Ch. 3, pp. 79–130 (Linux's process switch).

Concepts That Click

The Dispatcher

- The **dispatcher** is the kernel module that gives the CPU to the process the short-term scheduler selected.
- Its jobs: switch context, switch to user mode, jump to the right location in the user program.
- **Dispatch latency**: the time it takes to stop one process and start another.

Scheduler vs. dispatcher

The scheduler is the *policy* (which process?). The dispatcher is the *mechanism* (make it run). Every dispatch involves a context switch.

Silberschatz et al., Ch. 5 §5.1 (from p. 183) — previewed here, developed in Lecture 8.

Mode Switch Is Not a Context Switch

Mode switch

User → kernel → user for the *same* process (e.g. a `getpid` system call). Save/restore a few registers; no PCB state change.

Context (process) switch

The CPU moves to a *different* process. Full register save, PCB state update, queue move, memory-map switch.

- Every context switch includes mode switches; most mode switches cause no context switch.
- An interrupt can be handled and return to the interrupted process with no process switch at all.

Stallings, Ch. 3 §3.4, p. 135.

Context-Switch Cost Is Pure Overhead

- The system does no useful work while switching.
- Speed depends on the hardware: memory speed, number of registers, and special instructions (e.g. load/store all registers).
- Typical cost: a few microseconds — plus indirect cost from cold caches and TLB.
- Some processors provide multiple register sets: a switch just changes a pointer to the current set.

The rule

The more complex the OS and its memory management, the more context must be saved — and the more each switch costs.

Silberschatz et al., Ch. 3 §3.2, p. 110.

Worked Example: Overhead of Switching

Setup

A context switch takes $s = 5$ μs. Each process runs for a time slice of q before being switched out.

$$\text{overhead fraction} = \frac{s}{q + s}$$

Slice q	**Switches per second** $\approx 1/(q+s)$	**Overhead**
10 ms	≈ 99.95	$5/10005 \approx 0.05\%$
1 ms	≈ 995	$5/1005 \approx 0.50\%$
100 μs	≈ 9524	$5/105 \approx 4.76\%$
10 μs	≈ 66667	$5/15 \approx 33.3\%$

Shrink the slice and the CPU spends more of its life switching than working.

Worked Trace: P1 and P2 on One Core

1. $t = 0$: P1 (gcc) running; P2 waiting on the keyboard.
2. $t = 3$ ms: key pressed. Interrupt; P2 $\rightarrow$ `ready`. Mode switch only: P1 resumes.
3. $t = 10$ ms: P1's slice expires. Context switch #1: save PCB_1, P1 $\rightarrow$ `ready`; restore PCB_2, P2 $\rightarrow$ `running`.
4. $t = 10.2$ ms: P2 echoes the key, calls `read`. Context switch #2: P2 $\rightarrow$ `waiting`; P1 $\rightarrow$ `running`.

Count

Two context switches and three kernel entries (interrupt, timer, system call) in 10.2 ms. With $s = 5$ μs, switching cost 10 μs, about 0.1% of the interval.

Socratic Check 3

Question

A process calls `getpid`, and later calls `read` on a pipe that is empty. Which of the two calls causes a context switch?

- `getpid`: a mode switch only — the kernel answers and returns to the same process.
- `read` on an empty pipe: the process must block (`running` → `waiting`), so the kernel switches to another process.
- The test: does the *caller* keep the CPU when the kernel returns?

States, queues, schedulers, and the context switch are one machine.
Follow one process through all of them.

Concepts Click

P3's Life, End to End

1. Submitted to the batch queue: `new`, in the job queue.
2. Long-term scheduler admits it at night: `new` → `ready`, PCB on the ready queue.
3. Short-term scheduler picks it; dispatcher context-switches: `ready` → `running`.
4. It writes to the backup disk: `running` → `waiting`, on the disk queue; context switch away.
5. Memory is short; the medium-term scheduler swaps it out: `blocked` → `blocked/suspend`.
6. The write completes: `blocked/suspend` → `ready/suspend`; later swapped in: → `ready`.
7. Dispatched again, finishes, calls `exit`: `running` → `terminated`.

All three schedulers and both diagrams appear in one life.

Common Misconceptions

Wrong

- "A woken process starts running immediately."
- "Every system call is a context switch."
- "Every OS has a long-term scheduler."
- "The ready queue holds waiting processes."

Right

- It goes `waiting` $\rightarrow$ `ready` and queues.
- Most are mode switches; only blocking or preemption switches process.
- Time-sharing systems often have none.
- Waiting processes are on device queues.

Takeaways

- A process is in one of five states: `new`, `ready`, `running`, `waiting`, `terminated`; six transitions are legal. Suspension adds two more states.
- The kernel keeps PCBs on a **job queue**, a **ready queue**, and **device queues**; the queueing diagram shows every path.
- The **long-term** scheduler sets the degree of multiprogramming; the **short-term** scheduler picks the next process every few ms; the **medium-term** scheduler swaps.
- A **context switch** saves one PCB and restores another; it is pure overhead, $s/(q+s)$ of the CPU.
- A mode switch is not a context switch.

Concept That Click

Next: Operations on Processes

- Today: how the kernel labels, queues, schedules, and switches processes that already exist.
- Next lecture: how processes come to exist and how they end — `fork`, `exec`, `wait`, `exit`, and the process tree.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 3 §3.3 [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 3 §3.4 [3]; Bovet & Cesati, Ch. 3 [1].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 3 §3.1 (process states, p. 103), §3.2 (process scheduling: queues, schedulers, context switch, pp. 105–110).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 3 §3.2 (process states, two-, five-, and seven-state models, p. 111), §3.4 (process control, mode and process switching, p. 135).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 3 (processes and the process switch, pp. 79–130).

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Concepts That Click

References I


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click